

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

rehmang110_product_path = kagglehub.dataset_download('rehmang110/product')

print('Data source import complete.')
```

```
!pip install rembg
!pip install onnxruntime
# install the dependencies
!pip install diffusers transformers accelerate safetensors
# if VRAM is full then it is used so uninstall it and also
# if you do not uninstall it give error it only work when the vram is out of memory
#!pip uninstall xformers -y
!pip install controlnet_aux
```

[Show hidden output](#)

```
import torch
import numpy as np
from pathlib import Path
import shutil
from IPython.display import display
import cv2
# 2. Imports for text correction
import re
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer, pipeline

# image cleaning
from rembg import remove
from PIL import Image
import io
```

```
print(np.__version__)
print(torch.__version__)
```

```
1.26.4
2.6.0+cu124
```

```
if torch.cuda.is_available():
    device = "cuda"
    dtype= torch.float16

else:
    device= "cpu"
    dtype= torch.float32

print(f"Device : {device} , Data type : {dtype}")
```

```
Device : cuda , Data type : torch.float16
```

```
# Load correction model (this works)
model_id = "prithivida/grammar_error_correcter_v1"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForSeq2SeqLM.from_pretrained(model_id)
pipe = pipeline("text2text-generation", model=model, tokenizer=tokenizer)
# sending the model to device because if the gpu available then it could be a good use\
model.to(device)
```

[Show hidden output](#)

```
def clean_and_correct(prompt):
    prompt = prompt.strip().lower()
    prompt = re.sub(r'\s+', ' ', prompt)
    prompt = re.sub(r'^a-z0-9 ,.\'-]', '', prompt)

    corrected = pipe(prompt, max_new_tokens=32)[0]['generated_text'] # smaller token count
    return corrected
```

```
# Try it
raw_prompt = input("Enter your raw prompt: ")
final_prompt = clean_and_correct(raw_prompt)
print("🧼 Cleaned & Corrected Prompt:", final_prompt)
```

Enter your raw prompt: a good perfum bottle la on hablu
 🧼 Cleaned & Corrected Prompt: a good perfume bottle la on hablush.

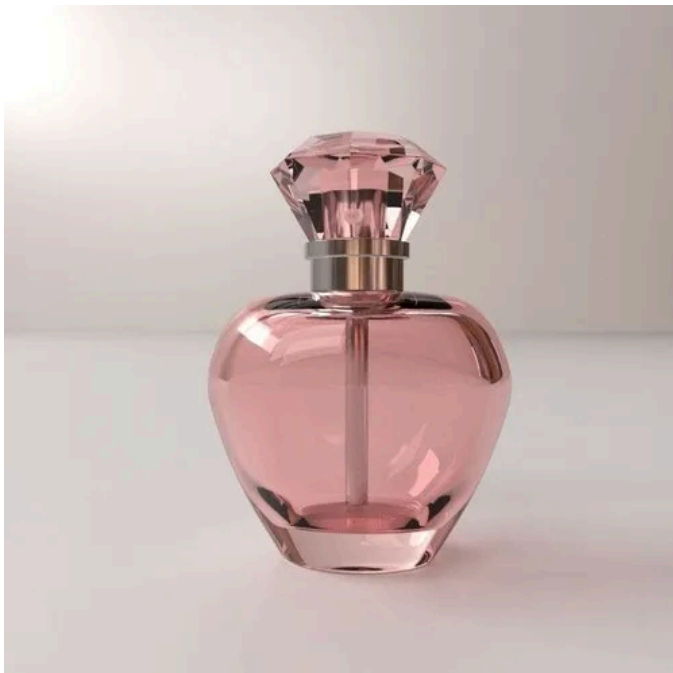
```
def clean_image(image_path, output_path="cleaned_image.jpg", output_size=(1024, 1024)):
    # Load and remove background
    input_image = Image.open(image_path).convert("RGBA")
    output_image = remove(input_image)

    # Resize and convert to RGB (removes alpha channel)
    output_image = output_image.resize(output_size).convert("RGB")

    # Save cleaned image
    output_image.save(output_path)
    return output_image
```

```
# code for movingt he inout to wroking directory
src_path= Path("/kaggle/input/product")
dest_path = Path("/kaggle/working/product_images")
dest_path.mkdir(parents=True, exist_ok=True)
for file in src_path.iterdir():
    shutil.copy(file, dest_path / file.name)
```

```
img_path =Path("/kaggle/working/product_images/perfume1.webp")
Image.open(img_path)
```



```
img = clean_image(str(img_path), output_path="cleaned_perfume.jpg")  
display(img)
```



Step-by-Step: Set Up SDXL with Hugging Face Diffusers

```
from diffusers import StableDiffusionXLPipeline  
import torch  
  
pipe = StableDiffusionXLPipeline.from_pretrained(  
    "stabilityai/stable-diffusion-xl-base-1.0",  
    torch_dtype=torch.float16,  
    use_safetensors=True,  
).to("cuda")  
  
pipe.enable_model_cpu_offload() # avoids high memory usage  
  
Loading pipeline components...: 0%|          | 0/7 [00:00<?, ?it/s]
```

```
prompt = "a bird flying ,and background is , beautiful sky with red clouds"
image = pipe(prompt=prompt).images[0]
image.save("generated_image.jpg")
Image.open("/kaggle/working/generated_image.jpg")
```

0%| | 0/50 [00:00<?, ?it/s]



IMAGE TO IMAGE DIFFUSER

```
from diffusers import StableDiffusionXLImg2ImgPipeline

pipe = StableDiffusionXLImg2ImgPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-refiner-1.0",
    torch_dtype=torch.float16,
    use_safetensors=True,
).to(device)
```

Loading pipeline components...: 0%| | 0/5 [00:00<?, ?it/s]

```
# Load your base image
init image = Image.open("/kaggle/working/cleaned perfume.jpg").convert("RGB").resize((1024, 1024))
```

```
# also prompt to guide the diffusion toward the direction
prompt = "a luxurious perfume bottle on a reflective table, studio lights, cinematic mood, ultra high quality"

# Lower strength keeps the original structure better
image = pipe(prompt=prompt, image=init_image, strength=0.8, guidance_scale=8.0).images[0]
image.save("stylized_output.jpg")
```

0% | 0/40 [00:00<?, ?it/s]

```
Image.open("/kaggle/working/stylized_output.jpg")
```



```
from diffusers import StableDiffusionControlNetPipeline, ControlNetModel
from diffusers.utils import load_image
from controlnet_aux import CannyDetector
import torch
import cv2
import numpy as np
```

```
# STEP 1: Load your input image (e.g., your perfume bottle)
```

```

image_path = "/kaggle/working/cleaned_perfume.jpg"
image = load_image(image_path).resize((512, 512))

# STEP 2: Convert to Canny edge map
canny = CannyDetector()
canny_image = canny(image)

# STEP 3: Load ControlNet model for Canny
controlnet = ControlNetModel.from_pretrained(
    "lllyasviel/sd-controlnet-canny", torch_dtype=dtype
)

# STEP 4: Load base SDXL (or SD1.5 if you're on low compute)
pipe = StableDiffusionControlNetPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5", # or use SDXL pipeline if your GPU is big enough
    controlnet=controlnet,
    safety_checker=None,
    torch_dtype=dtype,
).to("cuda")

# STEP 5: Define prompt and generate
prompt = "luxury perfume on glass table, dramatic lighting, cinematic background, studio setup"
output = pipe(prompt, image=canny_image, num_inference_steps=30)

# STEP 6: Save output
output.images[0].save("output_styled_perfume.jpg")

```

[Show hidden output](#)

```
Image.open("/kaggle/working/output_styled_perfume.jpg")
```



```

# Load pipeline
pipe = StableDiffusionXLImg2ImgPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-refiner-1.0",
    torch_dtype=torch.float16,
    use_safetensors=True,
).to(device)

# Prompt for enhancement

```

```

Loading pipeline components...: 0%|          | 0/5 [00:00<?, ?it/s]
0%|          | 0/15 [00:00<?, ?it/s]

```

```
imaaag_path = "/kaggle/working/product_images/perfume4.webp"  
Image.open(imaaag_path)
```



```
# Load your product image  
init_image = Image.open("/kaggle/working/product_images/perfume4.webp").convert("RGB")  
init_image = init_image.resize((512, 512))  
  
prompt = "luxury perfume on glass table, cinematic soft lighting, elegant studio background"  
  
# Generate image  
result = pipe(  
    prompt=prompt,  
    image=init_image,  
    strength=0.7, # 0.3 = subtle, 0.8 = strong change  
    guidance_scale=7.5,  
    num_inference_steps=50  
)  
  
result.images[0].save("styled_output.jpg")  
  
Image.open("/kaggle/working/styled_output.jpg")
```



```
from diffusers import StableDiffusionControlNetImg2ImgPipeline, ControlNetModel
from controlnet_aux import HEDdetector
from diffusers.utils import load_image
import torch
# 2. Create structure map using HED (more soft and rich than Canny)
hed = HEDdetector.from_pretrained("lllyasviel/Annotators")
hed_map = hed(image)

# 3. Load ControlNet (for HED) and Stable Diffusion
controlnet = ControlNetModel.from_pretrained(
    "lllyasviel/sd-controlnet-hed", torch_dtype=torch.float16
)
```

```
ControlNetHED.pth: 0% | 0.00/29.4M [00:00<?, ?B/s]
config.json: 0% | 0.00/920 [00:00<?, ?B/s]
diffusion_pytorch_model.safetensors: 0% | 0.00/1.45G [00:00<?, ?B/s]
```

```
# 1. Load your cleaned product image (512x512 preferred)
image_path = "/kaggle/working/cleaned_perfume.jpg"
image = load_image(image_path).resize((512, 512))

# Prompt
prompt = "luxury perfume bottle on glass table, cinematic lighting, beautiful background, product advertisement style"

# Generate
result = pipe(
    prompt=prompt,
    image=image,
    control_image=hed_map,
    num_inference_steps=30,
    strength=0.5,
    guidance_scale=9.0,
)

# Save
result.images[0].save("final_perfume_result.jpg")
Image.open("/kaggle/working/final_perfume_result.jpg")
```

0% | 0/15 [00:00<?, ?it/s]

